```
import asyncio
import os
import logging
import secrets
import sqlite3
import time
from threading import Thread
from typing import Set, Dict, Any, Optional, List
from flask import Flask
from aiogram import Bot, Dispatcher, types, F
from aiogram.filters import Command
from aiogram.types import (
    InlineKeyboardMarkup,
    InlineKeyboardButton,
    InlineQueryResultArticle,
    InputTextMessageContent
)

# --- Logging Configurations ---
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s",
    handlers=[
        logging.FileHandler("bot_metrics.log", encoding="utf-8"),
        logging.StreamHandler()
    ]
)

TOKEN: Optional[str] = os.getenv("BOT_TOKEN")
if not TOKEN:
    raise ValueError("BOT_TOKEN missing in Environment Variables.")

# Admin IDs ကို Env ကဖတ်မည်
ADMIN_IDS: List[int] = [int(x) for x in os.getenv("ADMIN_IDS", "7679480147").split(",")]

bot: Bot = Bot(token=TOKEN)
dp: Dispatcher = Dispatcher()

# --- Async Architecture Locks ---
db_lock: asyncio.Lock = asyncio.Lock()
game_lock: asyncio.Lock = asyncio.Lock()

games: Dict[str, Any] = {}
user_cooldowns: Dict[int, float] = {}

# --- SQLite Database Layer ---
DB_FILE = "database.db"
```

```
def init_db() -> None:
    """Database နှင့် Table များကို စတင်တည်ဆောက်ခြင်း။"""
    with sqlite3.connect(DB_FILE) as conn:
        cursor = conn.cursor()
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS users (
                user_id TEXT PRIMARY KEY,
                username TEXT,
                first_name TEXT,
                wins INTEGER DEFAULT 0,
                losses INTEGER DEFAULT 0,
                draws INTEGER DEFAULT 0,
                games_played INTEGER DEFAULT 0,
                win_streak INTEGER DEFAULT 0,
                coins INTEGER DEFAULT 100,
                last_seen TEXT
            )
        """)
        conn.commit()

init_db()

# --- DB Helper Functions ---
async def db_register_user(user_id: int, username: Optional[str], first_name: str) -> None:
    def _run():
        with sqlite3.connect(DB_FILE) as conn:
            cursor = conn.cursor()
            cursor.execute("""
                INSERT INTO users (user_id, username, first_name, last_seen)
                VALUES (?, ?, ?, datetime('now'))
                ON CONFLICT(user_id) DO UPDATE SET
                    username = excluded.username,
                    first_name = excluded.first_name,
                    last_seen = datetime('now')
            """, (str(user_id), username, first_name))
            conn.commit()
    async with db_lock:
        await asyncio.to_thread(_run)

async def db_update_stats(user_id: int, result: str) -> None:
    def _run():
        with sqlite3.connect(DB_FILE) as conn:
            cursor = conn.cursor()
            if result == "win":
                cursor.execute("""
                    UPDATE users SET wins = wins + 1, games_played = games_played + 1,
                    win_streak = win_streak + 1, coins = coins + 20 WHERE user_id = ?
                """, (str(user_id),))
```

```
            elif result == "loss":
                cursor.execute("""
                    UPDATE users SET losses = losses + 1, games_played = games_played + 1,
                    win_streak = 0 WHERE user_id = ?
                """, (str(user_id),))
            elif result == "draw":
                cursor.execute("""
                    UPDATE users SET draws = draws + 1, games_played = games_played + 1,
                    coins = coins + 5 WHERE user_id = ?
                """, (str(user_id),))
            conn.commit()
    async with db_lock:
        await asyncio.to_thread(_run)

async def db_get_profile(user_id: int) -> Optional[Dict[str, Any]]:
    def _run():
        with sqlite3.connect(DB_FILE) as conn:
            conn.row_factory = sqlite3.Row
            cursor = conn.cursor()
            cursor.execute("SELECT * FROM users WHERE user_id = ?", (str(user_id),))
            row = cursor.fetchone()
            return dict(row) if row else None
    async with db_lock:
        return await asyncio.to_thread(_run)

async def db_get_top() -> List[Dict[str, Any]]:
    def _run():
        with sqlite3.connect(DB_FILE) as conn:
            conn.row_factory = sqlite3.Row
            cursor = conn.cursor()
            cursor.execute("SELECT first_name, wins FROM users ORDER BY wins DESC LIMIT
10")
            return [dict(row) for row in cursor.fetchall()]
    async with db_lock:
        return await asyncio.to_thread(_run)

async def db_get_global_stats() -> Dict[str, int]:
    def _run():
        with sqlite3.connect(DB_FILE) as conn:
            cursor = conn.cursor()
            cursor.execute("SELECT COUNT(*) FROM users")
            total_users = cursor.fetchone()[0]
            return {"total_users": total_users}
    async with db_lock:
        return await asyncio.to_thread(_run)

# --- Utilities & Security Escaping ---
def escape_md(text: str) -> str:
```

```
    for char in ['_', '*', '`', '[']:
        text = text.replace(char, f"\\{char}")
    return text

# --- Flask Server Architecture ---
app: Flask = Flask(__name__)
PORT: int = int(os.getenv("PORT", "10000"))

@app.route('/')
def home() -> str:
    return "Bot Core Analytics Endpoint Active."

def run_flask() -> None:
    app.run(host='0.0.0.0', port=PORT)

# --- Safe Edit System ---
async def safe_edit(callback: types.CallbackQuery, text: str, reply_markup:
Optional[InlineKeyboardMarkup] = None) -> None:
    try:
        if callback.inline_message_id:
            await callback.bot.edit_message_text(
                text=text, inline_message_id=callback.inline_message_id,
                reply_markup=reply_markup, parse_mode="Markdown"
            )
        else:
            await callback.message.edit_text(text=text, reply_markup=reply_markup,
parse_mode="Markdown")
    except Exception as e:
        logging.error(f"Safe Edit Matrix Exception: {e}")

# --- Core Game Logic Mechanics ---
def get_turn_text(game: Dict[str, Any]) -> str:
    current_piece = game["turn"]
    c_theme = game["theme"][current_piece]
    p1 = game["creator"]
    p2 = game["opponent"]
    current_player_name = p1["name"] if p1["piece"] == current_piece else p2["name"]
    return (
        f"🎮 **Tic-Tac-Toe 4x4 ပွဲစဉ်**\n\n"
        f"🔴 {escape_md(p1['name'])} ({game['theme'][p1['piece']]})\n"
        f"🔵 {escape_md(p2['name'])} ({game['theme'][p2['piece']]})\n\n"
        f"⏳ **ယခုအလှည့်:** {escape_md(current_player_name)} ({c_theme}) ရဲ့အလှည့်"
    )

def create_board_keyboard(board: list, game_id: str, theme: Dict[str, str]) ->
InlineKeyboardMarkup:
    keyboard = []
    for r in range(4):
```

```
        row = []
        for c in range(4):
            symbol = board[r][c]
            display_text = theme[symbol] if symbol != ' ' else " "
            row.append(InlineKeyboardButton(text=display_text,
callback_data=f"move_{game_id}_{r}_{c}"))
        keyboard.append(row)
    keyboard.append([InlineKeyboardButton(text="🚪 Leave Game (ဘရိုးဖေးရန်)",
callback_data=f"leave_{game_id}")])
    return InlineKeyboardMarkup(inline_keyboard=keyboard)

def check_winner(board: list, player: str) -> bool:
    for i in range(4):
        if all(board[i][j] == player for j in range(4)) or \
           all(board[j][i] == player for j in range(4)):
            return True
    if all(board[i][i] == player for i in range(4)) or \
       all(board[i][3-i] == player for i in range(4)):
        return True
    return False

# --- Smart 4x4 Defensive Bot AI Logic ---
def get_ai_move(board: list) -> tuple:
    for r in range(4):
        for c in range(4):
            if board[r][c] == ' ':
                board[r][c] = 'O'
                if check_winner(board, 'O'): return r, c
                board[r][c] = ' '
    for r in range(4):
        for c in range(4):
            if board[r][c] == ' ':
                board[r][c] = 'X'
                if check_winner(board, 'X'):
                    board[r][c] = ' '
                    return r, c
                board[r][c] = ' '
    center_spots = [(1,1), (1,2), (2,1), (2,2)]
    for r, c in center_spots:
        if board[r][c] == ' ': return r, c
    for r in range(4):
        for c in range(4):
            if board[r][c] == ' ': return r, c
    return 0, 0

# --- Callback Handler with corrected indentation ---
@dp.callback_query(F.data.startswith("leave_"))
async def leave_callback(callback: types.CallbackQuery) -> None:
```

```
    _, game_id = callback.data.split("_")
    async with game_lock:
        if game_id not in games:
            return
        game = games[game_id]

    uid = callback.from_user.id
    if uid not in [game["creator"]["id"], game["opponent"]["id"]]:
        await callback.answer("သင်က ပွဲစဉ်ထဲက လူမဟုတ်ပါ။")
        return

    loser = game["creator"] if game["creator"]["id"] == uid else game["opponent"]
    winner = game["opponent"] if game["creator"]["id"] == uid else game["creator"]

    if game["opponent"]["id"] != 0:
        await db_update_stats(winner["id"], "win")
        await db_update_stats(loser["id"], "loss")

    await safe_edit(callback, f"🚪 **Leave Game! ပွဲစဉ်ကို လက်လျှော့လိုက်ပါပြီ။**\n\n😭
{escape_md(loser['name'])} သည် ပွဲအတွင်းမှ ထွက်ခွာသွားသဖြင့် အလိုအလျောက်
ရှုံးနိမ့်သွားပါသည်။\n🏆 အနိုင်ရရှိသူ: {escape_md(winner['name'])}", None)
    async with game_lock:
        if game_id in games:
            del games[game_id]

# --- Main Entry Point ---
async def main() -> None:
    await dp.start_polling(bot)

if __name__ == "__main__":
    t: Thread = Thread(target=run_flask)
    t.start()
    asyncio.run(main())
```